

บทที่ 3

ทฤษฎีและหลักการเกม 3 มิติ

ในการพัฒนาโปรแกรมเกม 3 มิติ จำเป็นอย่างยิ่งที่จะต้องมีความเข้าใจถึงทฤษฎีพื้นฐานที่จำเป็นสำหรับนำไปประยุกต์ใช้ โดยเฉพาะทฤษฎีทางด้านคอมพิวเตอร์กราฟิก ซึ่งเป็นพื้นฐานสำหรับการพัฒนาโปรแกรมเกม 3 มิติ

3.1 ระบบพิกัด 3 มิติ

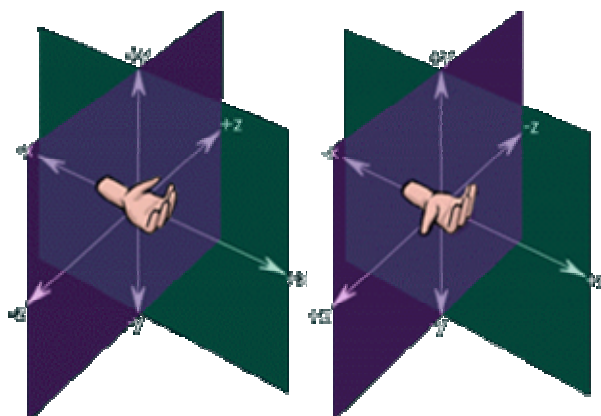
หมายถึงทิศทางของค่าในแนวแกน x , y และ z ของจุดกำเนิด (Origin) ในระบบสามมิติ ซึ่งจุดกำเนิดจะมีค่า x , y และ z เป็น 0, 0 และ 0 ตามลำดับ และโดยปกติแล้วทิศทางของค่า x จะเพิ่มขึ้นในทิศทางไปทางด้านขวา และลดลงไปทางด้านซ้ายของจุดกำเนิด ส่วนค่า y จะเพิ่มขึ้นไปทางด้านบน และลดลงไปทางด้านล่าง แต่ค่า z จะมีให้เลือกอยู่ 2 แบบ แบบแรกจะเพิ่มขึ้นในทิศทางเข้าไปในจอภาพ และแบบที่สองจะเพิ่มขึ้นในทิศทางออกจากจอภาพ โดยแบบแรกเรียกว่า Left-HandSystem ส่วนแบบที่สองจะเรียกว่า Right-HandSystem

3.1.1 ระบบพิกัดคาร์ทีเซียน (CartesianSystem)

เป็นระบบพิกัดที่ทำความเข้าใจได้ง่ายที่สุด เพราะเป็นระบบพิกัดที่คนส่วนใหญ่คุ้นเคย และมีใช้อย่างแพร่หลายที่สุด อีกทั้งยังเหมาะสมในการใช้สำหรับสร้างโปรแกรม ระบบพิกัดคาร์ทีเซียนประกอบด้วยสามแกนที่ตั้งฉากซึ่งกันและกันสำหรับกำหนดพิกัด โดยมักจะตั้งชื่อแกนนดังกล่าวให้เป็น X , Y และ Z ระบบพิกัดนี้โดยทั่วไปมักมีการกำหนดแนวแกนได้ 2 แบบคือ

1. ระบบพิกัดคาร์ทีเซียนแบบมือซ้าย (Left-handed CartesianSystem)
2. ระบบพิกัดคาร์ทีเซียนแบบมือขวา (Right-handed CartesianSystem)

โดยแสดงได้ดังภาพดังต่อไปนี้

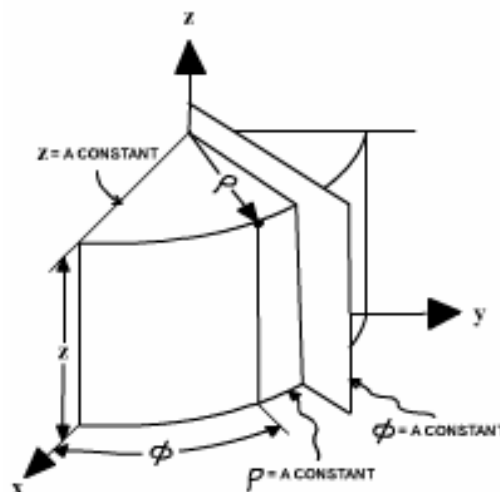


รูปที่ 3-1 แสดงระบบพิกัดคาร์ทีเซียนแบบมือซ้ายและแบบมือขวา

โดยทั่วไปไดเร็กเอ็กGraphics API นั้นระบบพิกัดที่ใช้จะเป็นระบบมือซ้าย ในการแสดงผลภาพ 3 มิติ นั้นจะต้องส่งเวอร์เท็กซ์อาร์เรย์ที่ต้องการแสดงผลเข้าไปใน Geometry Pipeline สุดท้ายก็จะได้ภาพ 2 มิติ ออกมาแสดงบนจอภาพ เหตุที่ต้องแปลงข้อมูลที่เป็น 3 มิติให้เป็น 2 มิติ นั้นเพราะหน้าจอของไม่สามารถ แสดงภาพ 3 มิติตรงๆ ได้จึงต้องทำการส่งข้อมูลเข้า Geometry Pipeline เพื่อให้ได้ผลลัพธ์เป็นภาพ 2 มิติ ที่ สอดคล้องกับข้อมูลนั้นบนมุมมองที่ต้องการ

3.1.2 ระบบพิกัดทรงกระบอก (Cylindrical System)

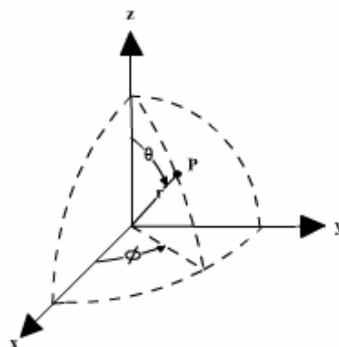
ในระบบพิกัดนี้ สามารถระบุพิกัดของจุดแต่ละจุดได้โดยใช้ระยะห่างระหว่างจุดนั้นๆ กับจุด กำเนิด (Origin) ซึ่งแทนด้วยสัญลักษณ์ ρ มุมระหว่างแกนอ้างอิงในแนวระดับ (มักสัมพันธ์กับแกน X และระนาบ XY ในระบบพิกัดคาร์ทีเซียน) กับเส้นตรงที่ลากจากจุดกำเนิดไปยังตำแหน่งของจุดนั้นๆ ใน ระนาบของจุดกำเนิดกับแกนอ้างอิงในแนวระดับ (ซึ่งสัมพันธ์กับระนาบ XY ในระบบพิกัดคาร์ทีเซียน) ซึ่งแทนด้วยสัญลักษณ์ ϕ และระดับความสูงของจุดนั้นๆ ซึ่งมักแทนด้วย Z ดังแสดงในรูป



รูปที่ 3-2 แสดงระบบพิกัดทรงกระบอก

3.1.3 ระบบพิกัดทรงกลม (Spherical System)

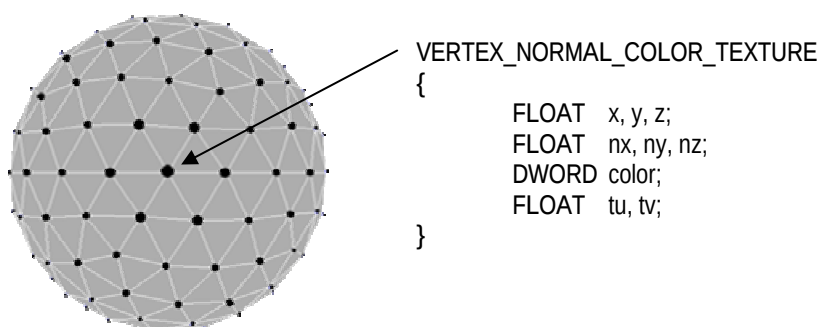
ในระบบพิกัดนี้ สามารถระบุพิกัดของจุดแต่ละจุดได้โดยใช้ระยะห่างระหว่างจุดนั้นๆ กับจุด กำเนิด ซึ่งแทนด้วยสัญลักษณ์ r มุมระหว่างแกนอ้างอิงในแนวระดับ (มักสัมพันธ์กับแกน X และระนาบ XY ในระบบพิกัดคาร์ทีเซียน) กับเส้นตรงที่ลากจากจุดกำเนิดไปยังตำแหน่งของจุดนั้นๆ ในระนาบของจุด กำเนิดกับแนวระดับ (ซึ่งสัมพันธ์กับระนาบ XY ในระบบพิกัดคาร์ทีเซียน) ซึ่งแทนด้วยสัญลักษณ์ ϕ และ มุมระหว่างแกนตั้งฉากแนวระดับ (แกน Z) กับเส้นตรงที่ลากจากจุดกำเนิดไปยังตำแหน่งของจุดนั้นๆ ดัง แสดงในรูป



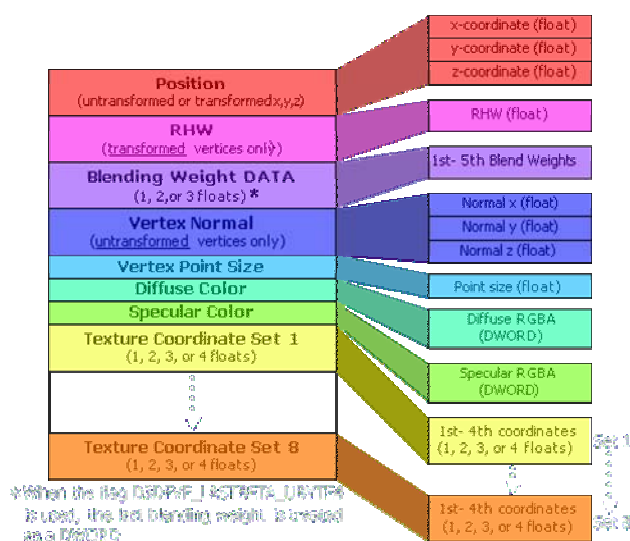
รูปที่ 3-3 แสดงระบบพิกัดทรงกลม

3.2 เวอร์เท็กซ์ (Vertex)

เวอร์เท็กซ์เป็นจุดพิกัดในระบบ 3 มิติ ซึ่งใช้ในการอ้างอิงตำแหน่ง ซึ่งในการประมวลผลทางด้านกราฟิกนั้นมักจะกำหนดสมบัติเพิ่มเติมให้กับเวอร์เท็กซ์ เช่น Color, Normal, Texture เป็นต้น ซึ่งสมบัติเหล่านี้เมื่อประกอบกับตำแหน่งของเวอร์เท็กซ์แล้วจะนำไปใช้ในการเก็บข้อมูลที่แทนแบบจำลอง 3 มิติได้ โดยแสดงได้ดังรูปตัวอย่าง



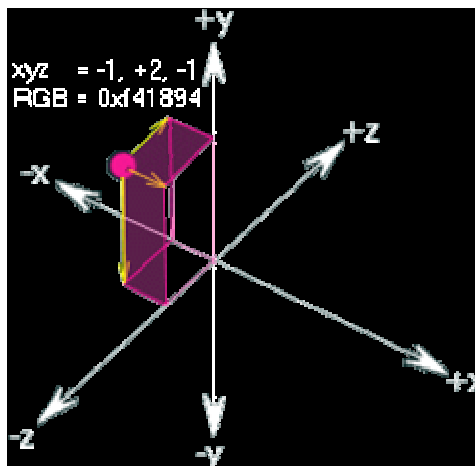
รูปที่ 3-4 แสดงตัวอย่างการนำเวอร์เท็กซ์มาใช้ในการเก็บข้อมูลที่แทนแบบจำลอง 3 มิติ



รูปที่ 3-5 โครงสร้างของเวอร์เท็กซ์ใน Direct3D

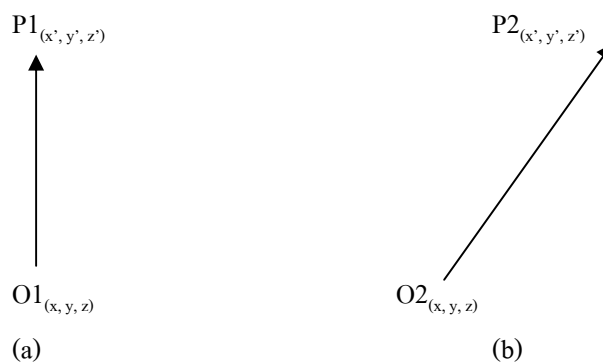
3.3 เวกเตอร์ (Vector)

เวกเตอร์เป็นปริมาณที่มีทั้งขนาดและทิศทาง เป็นองค์ประกอบทางคณิตศาสตร์ที่มีความสำคัญมากด้านการประยุกต์ใช้ในงานการประมวลผลกราฟิก ซึ่งจะต่างกับปริมาณสเกลาร์ตรงที่เวกเตอร์นั้นจะมีทิศทาง หรือกล่าวได้อีกอย่างหนึ่งว่าเวกเตอร์มีองค์ประกอบมาจากปริมาณสเกลาร์ n ตัว เพื่อแทนขนาดและทิศทางในระบบ n มิติ



รูปที่ 3-6 เวกเตอร์ของ RGB และ XYZ ใน Direct3D

มักจะแทนเวกเตอร์โดยใช้สัญลักษณ์ $\overrightarrow{OP}$ โดยหมายถึงเวกเตอร์นี้มีทิศทางจากจุด O ไปยังจุด P และมีขนาดเท่ากับระยะห่างระหว่างจุด O และจุด P



รูปที่ 3-7 แสดงตัวอย่างเวกเตอร์

หากทำการบวกเวกเตอร์ทั้งสองจะได้ค่าดังนี้

$$R = V1 + V2$$

$$R = (V1_x + V2_x, V1_y + V2_y, V1_z + V2_z)$$

เมื่อทำการคูณเวกเตอร์ด้วยปริมาณสเกลาร์ ซึ่งทำได้โดยการคูณปริมาณสเกลาร์กับทุกคอมโพเนนต์ของเวกเตอร์นั้น ๆ จะได้

$$V * s = (V_x * s, V_y * s, V_z * s)$$

สำหรับขนาด (Magnitude/length) ของเวกเตอร์ใด ๆ นั้น สามารถเขียนแทนด้วย $|V|$ ซึ่งสามารถหาได้โดยใช้กฎของพีทาโกรัส ได้สมการดังนี้

$$|V| = \sqrt{V_x^2 + V_y^2 + V_z^2}$$

เวกเตอร์ที่ขนานกัน (Parallel Vector) หมายถึง เวกเตอร์คู่ใด ๆ ที่มีทิศทางเดียวกันหรือทิศตรงข้ามกัน

เวกเตอร์ร่วมระนาบ (Coplanar Vector) หมายถึง เวกเตอร์ตั้งแต่ 3 ตัวขึ้นไปที่อยู่ในระนาบเดียวกัน

3.4 เมทริกซ์และทรานส์ฟอร์มเมชัน 3 มิติ

3.4.1 เมทริกซ์ (Matrix)

เมทริกซ์มิติ $m \times n$ ประกอบด้วยจำนวนจริงที่เขียนเรียงเป็นแถว (Row) m แถว และเขียนในแนวดิ่ง n หลัก (Column) โดยปิดล้อมจำนวนจริงเหล่านี้ด้วยเครื่องหมาย $[]$ หรือ $()$ จำนวนแต่ละจำนวนในเมทริกซ์ เรียกว่า สมาชิกของเมทริกซ์

$$\begin{bmatrix} 2 & 1 & 0 \\ 5 & 3 & 2 \end{bmatrix} \quad \text{เป็นเมทริกซ์มิติ } 2 \times 3 \quad \begin{bmatrix} 1 \\ 0 \\ 2 \\ 4 \end{bmatrix} \quad \text{เป็นเมทริกซ์มิติ } 4 \times 1$$

ถ้าเมทริกซ์ที่มีจำนวนแถวและจำนวนหลักเท่ากันเท่ากับ n จะเรียกเมทริกซ์นั้นว่า เมทริกซ์จัตุรัสมิติ n และเรียก $a_{11}, a_{12}, \dots, a_{nn}$ ว่าเป็นสมาชิกในแนวทแยงของ A เมื่อ A เป็นเมทริกซ์จัตุรัสมิติ n

a_{11}, a_{22}, a_{33} เป็นสมาชิกในแนวทแยงของ A หากเมทริกซ์จัตุรัส $A = [a_{ij}]$ ซึ่งมีสมาชิกทุกตัวนอกจากแนวทแยงเป็นศูนย์ทั้งหมด (นั่นคือ $a_{ij} = 0$ ถ้า $i \neq j$) เรียก A ว่าเป็น เมทริกซ์ทแยง (Diagonal Matrix)

ให้ $A = [a_{ij}]_{m \times n}$ และ $B = [b_{ij}]_{m \times n}$ เป็นเมทริกซ์ที่มีมิติเท่ากัน เรียก A เท่ากับ B ก็ต่อเมื่อ $a_{ij} = b_{ij}$ สำหรับ $1 \leq i \leq m, 1 \leq j \leq n$ และเขียนแทนด้วย $A = B$

ถ้า $A = [a_{ij}]_{m \times n}$ และ $B = [b_{ij}]_{m \times n}$ แล้วผลบวกของ A และ B เขียนแทนด้วย $A + B$ หมายถึง $C = [c_{ij}]_{m \times n}$ ซึ่ง

$$c_{ij} = a_{ij} + b_{ij} \quad (1 \leq i \leq m, 1 \leq j \leq n)$$

ให้ $A = [a_{ij}]_{m \times n}$ เป็นเมทริกซ์ และ k เป็นจำนวนจริงใดๆ แล้ว ผลคูณสเกลาร์ kA จะเป็นเมทริกซ์ $[ka_{ij}]_{m \times n}$ เช่น

$$\text{ให้ } A = \begin{bmatrix} 2 & 1 & 0 \\ 5 & 3 & 2 \end{bmatrix} \text{ และ } k = 3$$

$$\text{ดังนั้น } kA = \begin{bmatrix} 3(2) & 3(1) & 3(0) \\ 3(5) & 3(3) & 3(2) \end{bmatrix} = \begin{bmatrix} 6 & 3 & 0 \\ 15 & 9 & 6 \end{bmatrix}$$

ให้ $A = [a_{ij}]_{m \times p}$ และ $B = [b_{ij}]_{p \times n}$ แล้ว ผลคูณของ A และ B เขียนแทนด้วย AB หมายถึง

$$c = [a_{ij}]_{m \times n} \text{ โดย } c_{ij} = a_{i1}b_{1j} + a_{i2}b_{2j} + \dots + a_{ip}b_{jp}$$

$$c = \sum_{k=1}^p a_{ik} b_{kj} \quad (1 \leq i \leq m, 1 \leq j \leq n)$$

$$\begin{aligned} \text{เช่น } AB &= \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} \begin{bmatrix} b_{11} & b_{12} & b_{13} \\ b_{21} & b_{22} & b_{23} \end{bmatrix} \\ &= \begin{bmatrix} a_{11}b_{11} + a_{12}b_{21} & a_{11}b_{12} + a_{12}b_{22} & a_{11}b_{13} + a_{12}b_{23} \\ a_{21}b_{11} + a_{22}b_{21} & a_{21}b_{12} + a_{22}b_{22} & a_{21}b_{13} + a_{22}b_{23} \end{bmatrix} \end{aligned}$$

ผลคูณ AB ไม่มีหรือไม่นิยาม (Undefined) ถ้า A เป็นเมทริกซ์ขนาด $m \times p$ และ B เป็นเมทริกซ์ขนาด $q \times n$ เมื่อ $p \neq q$

การคูณเมทริกซ์นั้นไม่มีสมบัติการสลับที่ซึ่งหมายถึง $A \times B \neq B \times A$ เมื่อ A และ B เป็นเมทริกซ์จัตุรัส

เมทริกซ์เอกลักษณ์ (Identity Matrix) คือ เมทริกซ์จัตุรัสที่มีสมาชิกในแนวทแยงเป็น 1 ทั้งหมด ซึ่งเขียนแทนด้วย I หรือ I_n แทนเมทริกซ์เอกลักษณ์มิติ n เช่น

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

ถ้า A เป็นเมทริกซ์มิติ $m \times n$ แล้วจะได้ว่า $AI_n = A$ และ $I_m A = A$

B เป็นอินเวอร์สการคูณของเมทริกซ์ A (B เป็นอินเวอร์สของ A) เมื่อ B เป็นเมทริกซ์ซึ่ง $AB = BA = I$ โดยที่ A เป็นเมทริกซ์จัตุรัสใดๆ

ให้ A เป็นเมทริกซ์จัตุรัสมิติ n จะกล่าวว่า A มีตัวผกผัน (Invertible) หรือ A เป็นเมทริกซ์ซึ่งมิใช่เอกฐาน (Non-Singular Matrix) ถ้าสามารถหาเมทริกซ์จัตุรัส B ได้ ซึ่งทำให้

$$AB = I_n = BA$$

และเรียกเมทริกซ์ B ว่าเป็นอินเวอร์สการคูณ ของ A เขียนแทนด้วยสัญลักษณ์ A^{-1} นั่นคือ ถ้า A เป็นเมทริกซ์ซึ่งมิใช่เมทริกซ์เอกฐานมิติ n แล้ว จะได้ว่า

$$AA^{-1} = A^{-1}A = I_n$$

และถ้า A ไม่มีอินเวอร์ส แล้วจะเรียก A ว่าเป็นเมทริกซ์เอกฐาน (Singular Matrix)

3.4.2 Translation

เป็นการเคลื่อนย้ายตำแหน่งของวัตถุ ซึ่งใช้ Translation Matrix

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ T_x & T_y & T_z & 1 \end{bmatrix}$$

รูปที่ 3-8 Translation Matrix

3.4.3 Rotation

เป็นการหมุนวัตถุรอบแกน X, Y หรือ Z ซึ่งใช้ Rotation Matrix

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \theta & \sin \theta & 0 \\ 0 & -\sin \theta & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

รูปที่ 3-9 Rotation around X Axis Matrix

$$\begin{bmatrix} \cos \theta & 0 & -\sin \theta & 0 \\ 0 & 1 & 0 & 0 \\ \sin \theta & 0 & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

รูปที่ 3-10 Rotation around Y Axis Matrix

$$\begin{bmatrix} \cos \theta & \sin \theta & 0 & 0 \\ -\sin \theta & \cos \theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

รูปที่ 3-11 Rotation around Z Axis Matrix

3.4.4 Scaling

เป็นการย่อหรือขยายวัตถุ ซึ่งใช้ Scaling Matrix

$$\begin{bmatrix} S_x & 0 & 0 & 0 \\ 0 & S_y & 0 & 0 \\ 0 & 0 & S_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

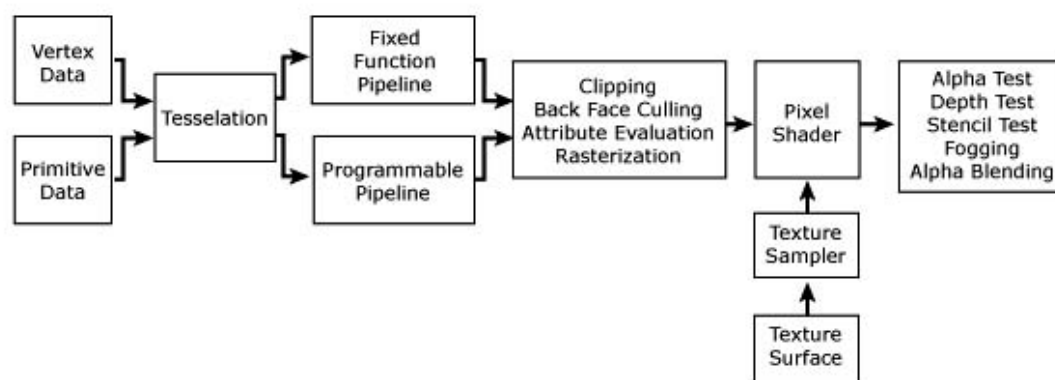
รูปที่ 3-12 Scaling Matrix

3.5 การแสดงผลภาพ 3 มิติด้วย Direct3D

3.5.1 สถาปัตยกรรมของ Direct3D

สถาปัตยกรรมของ Direct3D ประกอบด้วยหน้าที่การทำงานต่างๆ ซึ่งจะรวมกันทำงานแบบ Pipeline ดังที่แสดงตามรูป

Graphics Pipeline



รูปที่ 3-13 รูปแสดง Pipeline การทำงานของ Direct3D

3.5.2 ทรานส์ฟอร์มเมชัน

3.5.2.1 การแปรไปสู่พิกัดเวิลด์ (World Transformation)

ขั้นตอนนี้จะเป็นขั้นตอนในการแปลง Local ไปเป็น World ซึ่งเป็นตำแหน่งจริงๆ ในปริภูมิ 3 มิติ ซึ่งวิธีการนั้นจะทำการคูณตำแหน่งในเวอร์เท็กซ์ด้วย Transformation เมทริกซ์ต่าง ๆ เพื่อให้ได้ตำแหน่งที่ต้องการ

ในการแปลงเวอร์เท็กซ์จาก Local ให้เป็น World นั้นในบางครั้งถ้าการแปลงของมีความซับซ้อนมากอาจจะต้องใช้เมทริกซ์หลายตัวในการแปลงคูณต่อกันไป (Matrix Concatenation) เพื่อสร้างเมทริกซ์สุดท้ายสำหรับใช้เป็น Transformation เมทริกซ์ก่อนแล้วจึงจะเอามาคูณกับตำแหน่งในเวอร์เท็กซ์จริงเพื่อให้ได้ผลลัพธ์สุดท้ายก็ได้

การคูณเมทริกซ์กับตำแหน่งของเวอร์เท็กซ์จะเป็นดังต่อไปนี้

$$[x' y' z' 1] = [x y z 1] \begin{bmatrix} M_{11} & M_{12} & M_{13} & M_{14} \\ M_{21} & M_{22} & M_{23} & M_{24} \\ M_{31} & M_{32} & M_{33} & M_{34} \\ M_{41} & M_{42} & M_{43} & M_{44} \end{bmatrix}$$

$$x' = (x \times M_{11}) + (y \times M_{21}) + (z \times M_{31}) + (1 \times M_{41})$$

$$y' = (x \times M_{12}) + (y \times M_{22}) + (z \times M_{32}) + (1 \times M_{42})$$

$$z' = (x \times M_{13}) + (y \times M_{23}) + (z \times M_{33}) + (1 \times M_{43})$$

รูปที่ 3-14 การคูณเมทริกซ์กับตำแหน่งเวอร์เท็กซ์

จากรูป x, y, z คือ ตำแหน่งของเวอร์เท็กซ์พอยน์ทกับเมทริกซ์ดังรูปแล้ว จะได้ผลลัพธ์ เป็น x', y', z' ซึ่งเป็นผลลัพธ์ของการแปลงพิกัด

3.5.2.2 การแปรไปสู่พิกัดวิว (View Transformation)

ขั้นตอนนี้ใช้สำหรับแปลง World ให้สอดคล้องกับการเห็นของผู้สังเกต เนื่องจาก World บอกเพียงตำแหน่งจริงๆ ในพิกัด 3 มิติเท่านั้น ยังไม่มีการอ้างอิงจากผู้สังเกตเลย ดังนั้นจึงต้องมีการบอก ว่าผู้สังเกตอยู่ที่ไหน และมองไปทางไหน ขั้นตอนนี้มักใช้ในการปรับมุมมองของกล้อง โดยการนำวิวเมทริกซ์มาคูณกับ World จากการแปรไปสู่พิกัดเวิลด์

$$\begin{bmatrix} u_x & v_x & n_x & 0 \\ u_y & v_y & n_y & 0 \\ u_z & v_z & n_z & 0 \\ -(u \cdot c) & -(v \cdot c) & -(n \cdot c) & 1 \end{bmatrix}$$

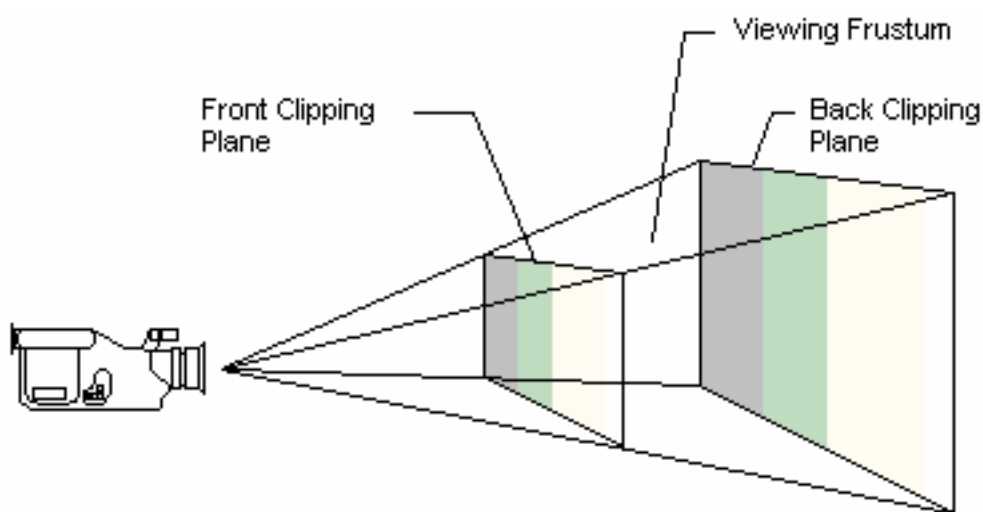
รูปที่ 3-15 การแปรไปสู่วิว

จากรูปวิวเมทริกซ์ประกอบด้วยเวกเตอร์ 3 ตัวบอกทิศทางของผู้สังเกตว่าหันหน้าไปทางใด ซึ่งมีเวกเตอร์ u , v , n บอกถึง Up, Right, และ View Direction ตามลำดับ และเวกเตอร์ c ซึ่งบอกตำแหน่งของผู้สังเกตในพิกัด World

วิวเมทริกซ์นั้นให้ข้อมูลที่จำเป็นในการปรับมุมมองของกล้องในเกมเอนจินได้ ซึ่งใช้ในการบอกตำแหน่งและทิศทางของกล้อง

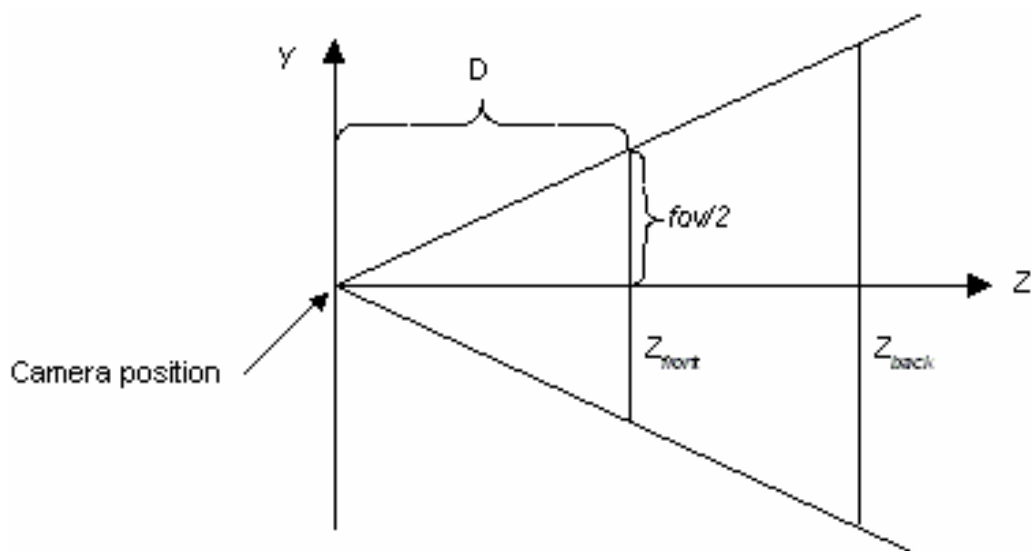
3.5.2.3 การแปรไปสู่วิวโปรเจกต์ชัน (Projection Transformation)

เมื่อได้ตำแหน่งซึ่งอ้างอิงกับผู้สังเกตเรียบร้อยแล้ว ขั้นตอนต่อไปก็คือการแปลง View มาเป็นพิกัด 2 มิติ ซึ่งสามารถนำมาวาดได้จริงๆ บนจอภาพ ซึ่งโดยทั่วไปแล้วต้องการให้ได้ภาพที่สมจริง คือ วัตถุที่อยู่ไกลจากสายตาดจะมีขนาดเล็กลง และวัตถุที่อยู่ใกล้สายตาดจะมีขนาดใหญ่ขึ้น ซึ่งเรียกว่า Perspective Projection Transformation ซึ่งเป็นการสร้างภาพฉายมาดกกระทบฉาก คล้ายๆ กับการฉายภาพจากโปรเจกเตอร์

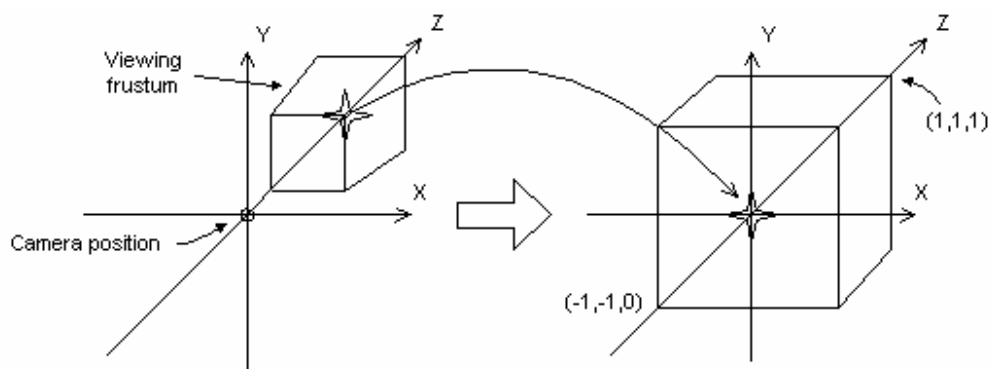


รูปที่ 3-16 Viewing Frustum

วัตถุที่อยู่ใน Viewing Frustum จะถูกนำมาแสดงผล เนื่องจากเป็นส่วนที่จอภาพสามารถเห็นได้ ส่วนวัตถุที่อยู่ภายนอก Viewing Frustum จะถูกตัดออกระหว่างการประมวลผลก่อนที่จะนำไปแสดงผล



รูปที่ 3-17 Viewing Frustum มองจากแนวแกน X



รูปที่ 3-18 การแปลงจาก Viewing Frustum (จากภาพด้านซ้าย) ไปเป็น Cuboid shape (จากภาพด้านขวา)
ซึ่งทำให้วัตถุที่อยู่ใกล้มีขนาดเล็กลง และวัตถุที่อยู่ไกลมีขนาดใหญ่ขึ้นตามสัดส่วนของระยะห่างจาก
สายตาผู้สังเกต

สามารถทำ Perspective Projection Transformation ได้โดยใช้ Perspective Projectionเมทริก
ดังนี้

$$\begin{bmatrix} w & 0 & 0 & 0 \\ 0 & h & 0 & 0 \\ 0 & 0 & Q & 1 \\ 0 & 0 & -QZ_{*} & 0 \end{bmatrix}$$

$$w = \cot\left(\frac{fov_w}{2}\right)$$

$$h = \cot\left(\frac{fov_h}{2}\right)$$

$$Q = \frac{Z_f}{Z_f - Z_n}$$

รูปที่ 3-19 Perspective Projection Matrix

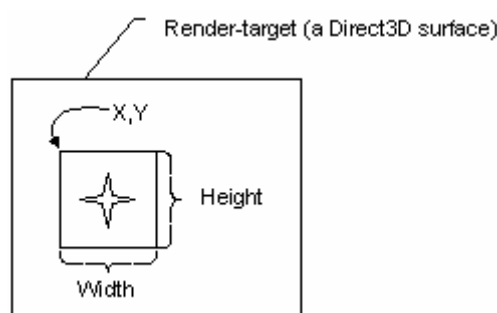
โดยทั่วไปแล้วกราฟิกเอนจินนั้นมักจะมีฟังก์ชันที่ใช้สำหรับการคำนวณหา Perspective Projection Matrix โดยให้กำหนดค่า Field of View (fov) ตามแนวความสูงของฉาก (โดยทั่วไปมักเป็นแกน Y) ค่า Aspect Ratio (w/h) ค่า Near Clipping Plane (Z_n) และค่า Far Clipping Plane (Z_f)

นอกจากการทำ Perspective Projection Transformation แล้วยังมีการทำ Projection Transformation แบบอื่นๆ อีก เช่น Isometric Projection Transformation, Oblique Projection Transformation เป็นต้น

3.5.3 Clipping และ Viewport Scaling

Viewport ในที่นี้หมายถึงพื้นที่สี่เหลี่ยมผืนผ้าที่ใช้ในการฉายภาพที่เกิดจากการแปรไปสู่วิกัดโปรเจกต์ชัน ซึ่งเป็นขั้นตอนสุดท้ายใน Geometry Pipeline และทำให้ได้ภาพฉายตกลงบน Viewport เพื่อทำการ Rasterization ต่อไปในการสร้างภาพเสมือนจริงบนหน้าจอ

Viewport อาจหมายถึงหน้าจอทั้งหน้าจอหรือบางส่วนของหน้าจอก็ได้ ทั้งนี้ขึ้นอยู่กับโปรแกรม ซึ่งโดยทั่วไปแล้วเกมมักจะแสดงผลแบบเต็มจอ แต่การใช้ Viewport อาจมีประโยชน์เมื่อต้องการการแสดงผล 3 มิติหลายหน้าต่าง



รูปที่ 3-20 Direct3D Viewport

โดยทั่วไปแล้วจะทำการขริบ (Clipping) สิ่งที่ไม่เห็นบนหน้าจอออกไป ในกรณีที่ Viewport แสดงถึงบางส่วนของหน้านั้น จะขริบส่วนของภาพที่อยู่นอกกรอบ Viewport ออกไป ทั้งนี้รวมถึงการ

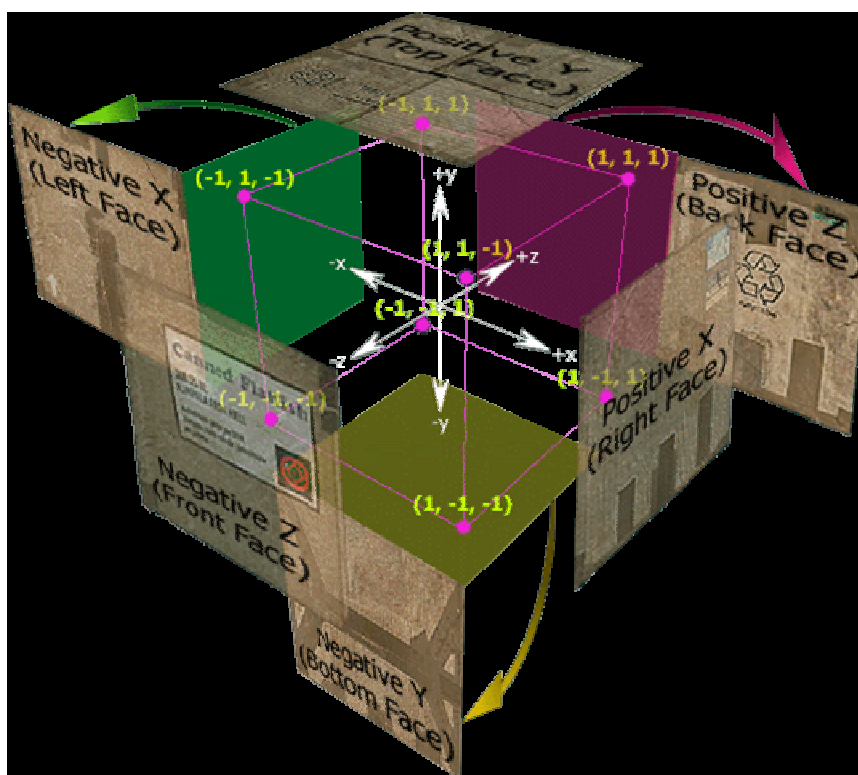
ขยับในแนวแกน Z ด้วย (จะตัดทอนส่วนของภาพที่อยู่ไกลเกิน Viewport ที่กำหนด หรือ ส่วนของภาพที่อยู่ด้านหลัง Near Clipping Plane ของ Viewing Frustum ออก)

กระบวนการนี้โดยมากมักจัดการโดยกราฟิกเอนจินโดยกำหนดค่าขอบเขตของ Viewport และระยะการตัดทอนตามแนวแกน Z ก็เพียงพอแล้ว

3.6 สเตป

3.6.1 โมเดลสเตป

โมเดลสเตป คือพิกัด (3D Coordinate) ที่ใช้ภายในโมเดลหรือวัตถุแต่ละอัน ซึ่งมีขอบเขตเป็นสี่เหลี่ยม โมเดลจะประกอบขึ้นมาจากรูปสามเหลี่ยมหลายๆ ชิ้น และรูปสามเหลี่ยมแต่ละชิ้นก็เกิดขึ้นจากเวอร์เท็กซ์จำนวน 3 จุด โดยแต่ละจุดก็จะอ้างอิงถึงจุดกำเนิด (Origin) จุดเดียวกัน (จุดกำเนิดจะมีค่า x, y, z เป็น 0, 0, 0) ซึ่งปกติแล้ว จุดกำเนิดนี้จะอยู่ที่กลางโมเดลแต่ละอัน ซึ่งพิกัดต่างๆ ที่โมเดลแต่ละอันใช้นี้จะเรียกว่า Local ขอบเขตของ โมเดลสเตป จะกว้าง, ยาว และสูงแค่ไหนก็ขึ้นอยู่กับขนาดของโมเดล จากรูป จะสร้างโมเดลเป็นกล่องกระดาษรูปทรงสี่เหลี่ยมหรือลูกบาศก์ ขนาดกว้าง 2 หน่วย, ยาว 2 หน่วย และสูง 2 หน่วย โดยจะมีจุด Origin อยู่ที่ใจกลางกล่อง ซึ่งโดยพื้นฐานของกล่องสี่เหลี่ยมแล้ว จะประกอบไปด้วยด้านสี่เหลี่ยม 6 ด้าน และจะทำให้ด้านแต่ละด้านของกล่องๆ นี้มีเท็กซ์เจอร์ไม่เหมือนกัน โดยภาพเท็กซ์เจอร์ทั้ง 6 ที่จะนำมา Map เข้ากับด้านแต่ละด้านนั้น จะถูกเรียงตามรูปแบบดังภาพด้านขวามือ ซึ่งวิธีเรียกด้านแต่ละด้านนี้จะใช้แกน x, y และ z เป็นตัวกำหนด โดยเรียงค่าตามลำดับในแนวแกน x, y และ z ดังนี้



รูปที่ 3-21 คำเรียกด้านของเท็กซ์เจอร์

1. ด้านที่อยู่ในแกน x ทางด้านบวก จะเรียกว่า ด้าน Positive X หรือด้านขวา (Right Face)
2. ด้านที่อยู่ในแกน x ทางด้านลบ จะเรียกว่า ด้าน Negative X หรือด้านซ้าย (Left Face)
3. ด้านที่อยู่ในแกน y ทางด้านบวก จะเรียกว่า ด้าน Positive Y หรือด้านบน (Top Face)
4. ด้านที่อยู่ในแกน y ทางด้านลบ จะเรียกว่า ด้าน Negative Y หรือด้านล่าง (Bottom Face)
5. ด้านที่อยู่ในแกน z ทางด้านบวก จะเรียกว่า ด้าน Positive Z หรือด้านหลัง (Back Face)
6. ด้านที่อยู่ในแกน z ทางด้านลบ จะเรียกว่า ด้าน Negative Z หรือด้านหน้า (Front Face)

และด้านล่างนี้เป็นภาพเท็กซ์เจอร์จำนวน 6 ภาพ ซึ่งแต่ละภาพมีขนาด 128 x 128 เมื่อนำมาเรียงต่อกันตามรูปแบบข้างบน ก็จะดูเหมือนกล่องกระดาษที่ถูกคลี่ออกมา หรือดูเป็นกระดาษที่ยังไม่ได้ถูกพับให้เป็นกล่อง



รูปที่ 3-22 การคลี่เท็กซ์เจอร์

จะใช้ขนาดเท็กซ์เจอร์เท่าไรก็ได้ แต่ถ้ายังขนาดใหญ่เกินไป ก็จะทำให้เฟรมเรตลดลงด้วย (จะทำให้ภาพกระตุก) บางทีก็ต้องดูด้วยว่า การ์ด 3D รองรับขนาดเท็กซ์เจอร์สูงสุดได้แค่ไหน เช่น ถ้าเป็นการ์ด Voodoo Graphics หรือ Voodoo รุ่นแรก จะ Support ที่ 256 x 256 Texel แต่ถ้าเป็นการ์ด Voodoo5 5500 ที่ใช้อยู่ปัจจุบัน จะ Support ได้ถึง 2048 x 2048 Texel ซึ่งการจะรองรับได้ขนาดไหนนั้น จะอยู่ที่ขนาดของหน่วยความจำของการ์ด 3D ด้วย (Voodoo Graphics มี RAM ขนาด 4 MB ส่วน Voodoo5 5500 มี 64 MB)

เกม 3D จะมีการระบุว่าการ์ด 3D ที่มี RAM เท่าไรเป็นอย่างต่ำ เพื่อที่จะได้รองรับขนาดเท็กซ์เจอร์สูงสุดที่ใช้ในการเขียนเกมนั้นๆ ได้ เพราะการ Map รูปภาพเท็กซ์เจอร์ลงไปบนโมเดล จะต้องมีการ Copy รูปภาพจากหน่วยความจำปกติ ไปไว้ในหน่วยความจำของการ์ด 3D ถ้ามีการเปลี่ยนเท็กซ์เจอร์อันใหม่ ก็จะต้อง Copy ลงไปใหม่ ดังนั้นถ้าโปรแกรมที่เขียนมีการเปลี่ยนเท็กซ์เจอร์เพื่อใช้ในการ Map บ่อยๆ จะทำให้ Frame Rate ลดลง วิธีที่ถูกต้องคือ ควรวาดภาพโดยเรียงตามเท็กซ์เจอร์ที่ใช้ คือ Group รูป

สามเหลี่ยมหรืออาจจะ Group จุดเวอร์เท็กซ์ที่เท็กซ์เจอร์เดียวกันเข้าด้วยกัน ตัวอย่างเช่น ถ้าต้องการวาดกล่องสี่เหลี่ยมที่เหมือนกันจำนวน 10 กล่องลงไปในฉาก โดยแต่ละกล่องมีการใช้เท็กซ์เจอร์จำนวน 6 ภาพที่ต่างกันสำหรับใช้ Map ลงไปที่ด้านแต่ละด้านของกล่อง ถ้าใช้วิธี Group ตามจำนวนกล่อง หรือวาดทีละกล่อง คือวาดกล่องแรกจนเสร็จ แล้ววาดกล่องต่อไป ซึ่งการทำแบบนี้จะต้องมีการเปลี่ยนเท็กซ์เจอร์ถึง 60 ครั้ง (ทวนรอบนับจำนวนกล่อง 10 รอบ และแต่ละรอบมีการวนรอบเปลี่ยนเท็กซ์เจอร์อีก 6 รอบ รวมจำนวนรอบของ Loop ทั้งหมดเป็น 10×6 ซึ่งเท่ากับ 60)

แต่ถ้าใช้วิธีนี้ กล่องไหนมีด้านที่ต้องใช้เท็กซ์เจอร์ที่กำลังใช้อยู่ ก็วาดด้านนั้นให้เสร็จก่อน แล้วจึงเปลี่ยนเท็กซ์เจอร์เป็นด้านต่อไป และวาดด้านอื่นของกล่องทั้งหมดที่ใช้เท็กซ์เจอร์อันใหม่นี้ ทำอย่างนี้ไปเรื่อยๆ จะมีการเปลี่ยนเท็กซ์เจอร์เพียง 6 ครั้งเท่านั้น ทั้งๆ ที่ใช้จำนวนรอบรวมของการวนรอบเท่ากัน (วนรอบนับจำนวนเท็กซ์เจอร์ 6 รอบ และแต่ละรอบมีการวนรอบภายในเปลี่ยนกล่องที่จะวาด 10 รอบ รวมจำนวนรอบของ Loop ทั้งหมดเป็น 6×10 ซึ่งเท่ากับ 60) ซึ่งในตัวอย่างโปรแกรมจะวาดโดย Group ด้านที่ใช้เท็กซ์เจอร์เดียวกันเข้าด้วยกัน

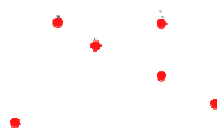
ตามคำแนะนำในคู่มือโคเรอ์เอ็ค SDK นั้นบอกว่า ควรใช้ขนาดเท็กซ์เจอร์ที่เล็กที่สุดเท่าที่จะทำได้นอกจากนี้ควรใช้เท็กซ์เจอร์ที่เป็นรูปสี่เหลี่ยมจัตุรัสที่มีความกว้างและความสูงเท่ากัน (ปกติอาจจะใช้แบบไม่เท่ากันก็ได้ เช่น 256×128 หรือ 100×50 เป็นต้น) โดยขนาดของเท็กซ์เจอร์ที่เหมาะสมที่สุดก็คือ 256×256 และควรพยายามทำให้เท็กซ์เจอร์เล็กหลายๆ อัน มารวมกันให้เป็นขนาด 256×256 เช่น สมมุติว่ามีเท็กซ์เจอร์ขนาด 128×128 จำนวน 4 รูป ก็ควรนำรูปทั้ง 4 มาต่อกันให้เป็นขนาด 256×256

สี่เหลี่ยมที่มองเห็นจาก Direct3D จะต้องประกอบขึ้นมาจากรูปสามเหลี่ยม 2 รูป (การ์ด 3D ส่วนมากจะ Render หรือทำงานกับรูปสามเหลี่ยม, เส้นตรง และจุดเป็นหลัก) และสามเหลี่ยมแต่ละรูปก็ประกอบขึ้นมาจากเวอร์เท็กซ์จำนวน 3 จุด แต่ไม่จำเป็นว่า รูปสี่เหลี่ยม 1 รูปที่เกิดจากรูปสามเหลี่ยม 2 รูปมาประกบกันจะมีเวอร์เท็กซ์จำนวน 6 จุดเสมอไป ขึ้นอยู่กับ Draw Primitive ที่จะใช้ ซึ่ง Draw Primitive ก็คือ "การวาดรูปทรงต่างๆ จากการเรียงของเวอร์เท็กซ์แต่ละจุด" โดยรูปแบบของ Draw Primitive ใน Direct3D จะมีอยู่ 6 แบบดังนี้

สมมุติว่าใช้เวอร์เท็กซ์จำนวน 6 จุด ในการวาด Primitive แต่ละแบบ โดยเวอร์เท็กซ์จะเรียงกันดังนี้

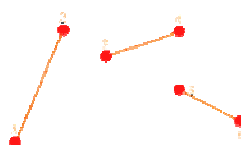
- Vertex จุดที่ 1 = $(-5.0f, -15.0f, 0.0f)$
- Vertex จุดที่ 2 = $(0.0f, 5.0f, 0.0f)$
- Vertex จุดที่ 3 = $(3.0f, 0.0f, 0.0f)$
- Vertex จุดที่ 4 = $(10.0f, 5.0f, 0.0f)$
- Vertex จุดที่ 5 = $(10.0f, -5.0f, 0.0f)$
- Vertex จุดที่ 6 = $(15.0f, -10.0f, 0.0f)$

1. Point List จะวาดจุด (Point) ตามลำดับการเรียงของเวอร์เท็กซ์แต่ละจุด (1 Point ต่อ 1 Vertex) ซึ่งค่าคงที่ที่ใช้ในกรณีนี้จะเป็น D3DPT_POINTLIST และผลลัพธ์จะเป็นไปตามรูปด้านล่างนี้



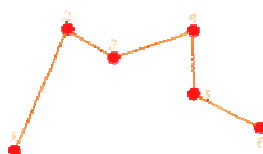
รูปที่ 3-23 Point List

2. Line List จะวาดเส้นตรง (Line) เรียงตามลำดับการเรียงของเวอร์เท็กซ์ทุกๆ 2 จุด โดยเส้นแต่ละเส้นจะแยกออกจากกัน (1 Line ต่อ 2 Vertex) ซึ่งค่าคงที่ที่ใช้ในกรณีนี้เป็น D3DPT_LINELIST



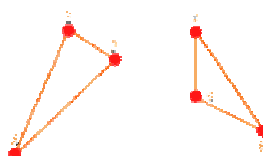
รูปที่ 3-24 Line List

3. Line Strip จะวาดเส้นตรงเรียงตามลำดับการเรียงของเวอร์เท็กซ์โดยที่เวอร์เท็กซ์ที่ 2 ของ Line ที่เพิ่งวาดไปจะกลายเป็นเวอร์เท็กซ์จุดที่ 1 ของ Line ถัดไป คือใช้เวอร์เท็กซ์ร่วมกัน 1 จุด ผลลัพธ์ที่ได้คือ Line ที่ต่อกันไปเรื่อยๆ ซึ่งค่าคงที่ที่ใช้ในกรณีนี้จะเป็น D3DPT_LINESTRIP



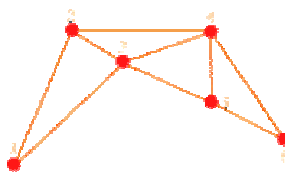
รูปที่ 3-25 Line Strip

4. Triangle List จะวาดสามเหลี่ยม (Triangle) เรียงตามลำดับการเรียงของเวอร์เท็กซ์ทุกๆ 3 จุด โดยสามเหลี่ยมแต่ละรูปจะแยกออกจากกัน (1 Triangle ต่อ 3 Vertex) ซึ่งค่าคงที่ที่ใช้ในกรณีนี้จะเป็น D3DPT_TRIANGLELIST



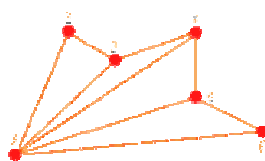
รูปที่ 3-26 Triangle List

5. Triangle Strip จะวาดสามเหลี่ยมเรียงตามลำดับการเรียงของเวอร์เท็กซ์โดยเวอร์เท็กซ์ที่ 2 และ 3 ของรูปสามเหลี่ยมอันที่เพิ่งวาดไป จะกลายเป็นเวอร์เท็กซ์ที่ 1 และ 2 ของรูปสามเหลี่ยมรูปถัดไป คือใช้เวอร์เท็กซ์ร่วมกัน 2 จุด ผลลัพธ์ที่ได้ก็คือ รูปสามเหลี่ยมที่ต่อกันไปเรื่อยๆ (ถ้าจะใช้วิธีนี้ ควรจะเรียงเวอร์เท็กซ์แบบซิกแซก) ซึ่งค่าคงที่ที่ใช้ในกรณีนี้จะเป็น D3DPT_TRIANGLESTRIP



รูปที่ 3-27 Triangle Strip

6. Triangle Fan จะวาดสามเหลี่ยมเรียงตามลำดับการเรียงของเวอร์เท็กซ์โดย มีเวอร์เท็กซ์แรกสุดเป็นเวอร์เท็กซ์ที่ 1 ของสามเหลี่ยมทุกรูป และเวอร์เท็กซ์ที่ 3 ของรูปสามเหลี่ยมอันที่เพิ่งวาดไป จะกลายเป็นเวอร์เท็กซ์ที่ 2 ของรูปสามเหลี่ยมรูปถัดไป ผลลัพธ์ก็คือ จะได้รูปสามเหลี่ยมที่เรียงตัวต่อกันเป็นรูปที่คล้ายพัด ซึ่งค่าคงที่ที่ใช้ในกรณีนี้จะเป็น D3DPT_TRIANGLEFAN



รูปที่ 3-28 Triangle Fan

เหตุที่ต้องมี Draw Primitive หลายๆ แบบให้เลือกใช้ก็เพื่อประหยัดหน่วยความจำ และเพิ่ม Frame Rate เพราะยังใช้เวอร์เท็กซ์น้อยเท่าไร ก็ยิ่งดีเท่านั้น ยกตัวอย่างรูปกล่องสี่เหลี่ยมที่อธิบายไว้แล้ว แต่ละด้านจะเป็นรูปสี่เหลี่ยม ซึ่งเกิดจากรูปสามเหลี่ยมสองรูปมาประกบกัน ถ้าจะวาดกล่องโดยใช้ Draw Primitive เป็น Triangle List จะต้องใช้เวอร์เท็กซ์จำนวน 36 จุด เพราะ สามเหลี่ยมแต่ละรูปเกิดขึ้นมาจากเวอร์เท็กซ์จำนวน 3 จุด ดังนั้นจึงต้องใช้เวอร์เท็กซ์ 6 จุดต่อการวาดรูปสี่เหลี่ยม 1 รูป และเนื่องจากกล่องมี 6 ด้าน ดังนั้นต้องใช้เวอร์เท็กซ์ทั้งสิ้น 6×6 ซึ่งเท่ากับ 36 จุด แต่ถ้าใช้ Draw Primitive แบบ Triangle Strip จะใช้เวอร์เท็กซ์แค่ 4 จุดต่อ 1 ด้านเท่านั้น รวมแล้วก็ 4×6 ซึ่งเท่ากับ 24 จุด

สำหรับ เมธอด ที่ใช้ในการวาดรูปก็คือ IDirect3DDevice8::DrawPrimitive() โดยมีพารามิเตอร์ตัวแรกเป็นค่าคงที่ของ Draw Primitive ที่จะใช้ (ให้ดูค่าคงที่ในหัวข้อ Draw Primitive ทั้ง 6 แบบข้างบน) Parameter ตัวที่สองจะเป็นตำแหน่งของเวอร์เท็กซ์ที่จะเริ่มวาดรูปสามเหลี่ยม และ Parameter ตัวสุดท้ายจะบอกให้รู้ว่า จะให้วาดสามเหลี่ยมจำนวนกี่รูป จากตัวอย่างโปรแกรม จะทำ Loop ไล่ตั้งแต่เท็กซ์เจอร์รูปแรกไปจนถึงรูปที่ 6 ที่ต้องทำ Loop ก็เพราะ แต่ละด้านใช้เท็กซ์เจอร์ต่างกัน แล้วใช้ เมธอด IDirect3DDevice8::SetTexture() เพื่อสั่งให้ Load ภาพเท็กซ์เจอร์ที่จะใช้ Map ด้านนั้นๆ เข้าไปไว้ในหน่วยความจำของการ์ด 3D แล้วก็วาดด้านตามการเรียงเวอร์เท็กซ์ตามที่กำหนดไว้ในตัวแปร g_vtVertex

3.6.2 World Space

World Space นั้น ถ้าพูดง่ายๆ ก็คือโลก 3 มิติ มันจะมีขนาดใหญ่กว่าโมเดลสเปซเพราะโมเดลต่างๆ ที่สร้างจะถูกแปลงให้มาปรากฏอยู่ใน World Space โดยหลังจากที่โมเดลได้ถูกนำมาใส่ไว้ใน World Space นี้แล้ว พวกมันจะมีจุด Origin เป็นจุดเดียวกัน สำหรับขนาดความกว้างของ World Space จะเป็น 2 ยกกำลัง 32 และความสูงจะเป็น 2 ยกกำลัง 32 สำหรับความลึกก็ขึ้นอยู่กับหน่วยความจำในแนวลึก (Depth Buffer หรือบางทีก็เรียก Z Buffer) ใน World Space นี้ สามารถทำอะไรกับโมเดลก็ได้ เช่น ย้ายตำแหน่ง (Translation), หมุน (Rotation) และขยายหรือลดขนาด (Scaling) เป็นต้น

3.6.3 Camera Space

เป็นมุมมองของกล้อง หรือดวงตา เหมือนกับกรณีที่มีมองไปที่สิ่งต่างๆ รอบๆ ตัว ซึ่งจะเห็นเฉพาะสิ่งที่ปรากฏอยู่ในสายตาเท่านั้น สิ่งที่อยู่ในสายตานี้แหละเรียกว่า Camera Space ดังนั้นสรุปได้ว่า Camera Space จะเป็นพื้นที่แบบสามมิติที่ดึงมาจากบางส่วนของ World Space

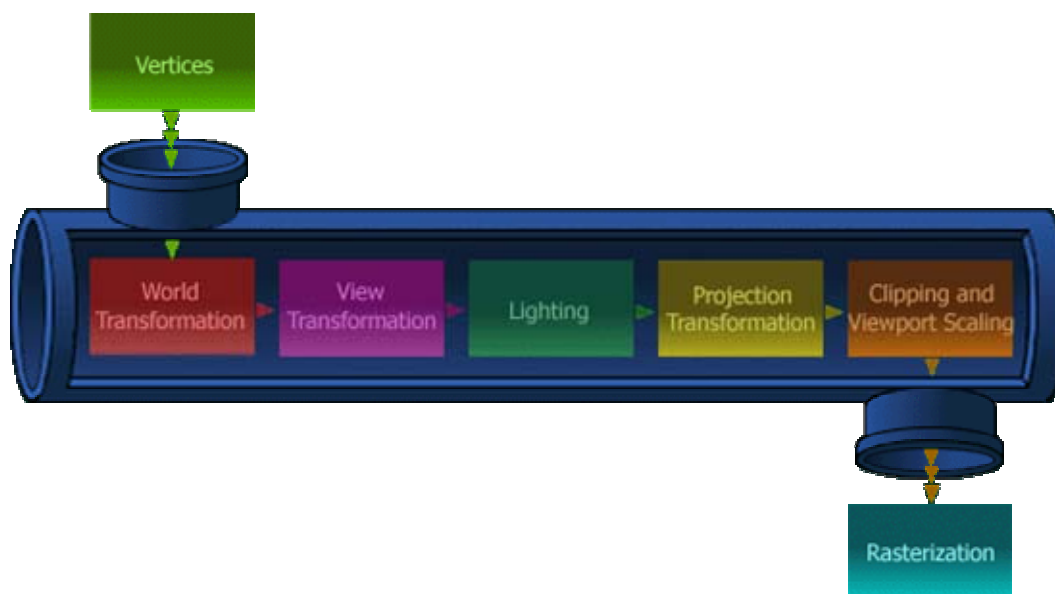
3.6.4 Projection Space

Screen Space คือพื้นที่สองมิติแบนๆ แบบภาพที่อยู่บนจอภาพ โดยพื้นที่นี้จะถูกแปลงมาจาก Projection Space เพื่อเตรียมเอาไปแสดงให้เห็นบนจอภาพ

3.6.5 Transformation and Lighting (T&L) และ Rasterization

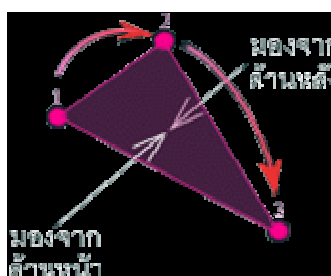
Transformation ก็คือขั้นตอนในส่วนของการคำนวณเพื่อแปลงหรือตำแหน่งของ Vertex ต่างๆ ที่ประกอบขึ้นเป็นโมเดลที่อยู่ใน Model Space ให้เป็นใน World Space (เรียกขั้นตอนนี้ว่า World Transformation) หลังจากนั้นก็ทำขั้นตอนการคำนวณเพื่อแปลง ตำแหน่งของ Vertex ต่างๆ บางส่วนของ World Space ให้ไปเป็นใน Camera Space (เรียกขั้นตอนนี้ว่า View Transformation) หลังจากนั้น จะเป็นการทำ Lighting โดยมันจะคำนวณเพื่อใส่สีต่างๆ ให้กับ Vertex เช่นสีจาก Texel ของเท็กซ์เจอร์ที่นำมา Map, สีของจุด Vertex นั้นๆ เอง หรือสีจากแหล่งกำเนิดแสงต่างๆ เป็นต้น

ขั้นต่อไปก็จะแปลงสิ่งที่อยู่ใน Camera Space ให้ไปอยู่ใน Projection Space (เรียกขั้นตอนนี้ว่า Projection Transformation) โดยที่ขั้นตอนนี้จะมีการคำนวณตำแหน่งและขนาดของโมเดลให้เพี้ยนไปจากเดิม เพื่อที่ว่าเมื่อถูกแปลงให้เห็นเป็นภาพ 2 มิติแล้ว จะได้เห็นเป็นแบบ Perspective คือสิ่งที่อยู่ใกล้ตา จะใหญ่กว่าสิ่งที่อยู่ไกลออกไป ซึ่งจะทำให้มองเหมือนว่า ภาพ 2 มิตินั้นดูมีความลึก ซึ่งการเขียนโปรแกรมเพื่อทำงานในส่วน Transformation ทั้ง 3 แบบที่กล่าวมานี้จะต้องใช้เมทริก (ซึ่งจะอธิบายเรื่องเมทริกในหัวข้อถัดไป)



รูปที่ 3-29 ขั้นตอนการทำ Transformation

เมื่อทำงานในส่วน Transformation and Lighting เสร็จแล้ว ยังมีการทำงานอันหนึ่งแทรกก่อนจะไปถึงการทำงานในส่วน Rasterization นั่นคือ การตัดส่วนของโมเดลที่ถูกโมเดลอื่นบัง หรือตัดส่วนที่อยู่ด้านหลังออกไปด้วยแล้วก็ลดหรือขยายขนาดโมเดลต่างๆ ให้พอดีกับขนาดของ Resolution ของหน้าจอที่ใช้ (เรียกขั้นตอนนี้ว่า Clipping and Viewport Scaling) ซึ่งขั้นตอนส่วนนี้จะช่วยลดการทำงานของขั้นตอนการ Rasterization คือไม่ต้องไปสนใจสิ่งที่มองไม่เห็นนั่นเอง จริงๆ แล้วการ Clipping และ Viewport Scaling นี้จะทำก่อน T&L ก็ได้ แต่ใน Direct3D มันมาทำตอนนี้ ถึงตอนนี้ เรื่องการเรียงตำแหน่ง Vertex ที่จะใช้การวาดรูปหรือ Draw Primitive จะเกี่ยวกับการตัดส่วนที่อยู่ด้านหลังที่ว่านี้(ด้านหลังของรูปสามเหลี่ยมจะไม่ถูก Render) ซึ่งโดย Default แล้ว Direct3D จะถือว่า ด้านหลังคือด้านที่ Vertex เรียงกันแบบทวนเข็มนาฬิกา (D3DRS_CULLMODE เป็น D3DCULL_CCW) ดังนั้น ต้องเรียง Vertex แบบตามเข็มนาฬิกา (ตามเข็มนาฬิกาจะเรียงจากล่างขึ้นบนแล้วไปทางขวา หรือจากบนลงล่างขวา ส่วนทวนเข็มนาฬิกาจะเรียงกลับกัน คือล่างขึ้นบนแล้วไปทางซ้าย หรือจากบนลงล่างซ้าย) จะสังเกตเห็นเหตุการณ์ที่ไม่ Render ด้านที่อยู่ข้างหลังได้โดยการรันโปรแกรมตัวอย่างแล้วใช้ปุ่ม S เลื่อนกล้องเข้ามาใกล้ๆ จนเข้าไปอยู่ในกล้อง จะเห็นว่ามันมีคสนิท



รูปที่ 3-30 ด้านของ Draw Primitive

ขั้นสุดท้ายจะเป็นการทำ Rasterization โดยตัวที่ทำหน้าที่นี้จะเรียกว่า Rasterizer จะแปลงสิ่งที่อยู่ใน Projection Space ให้ไปอยู่ใน Screen Space คือแปลงพิกัดของ Texel ต่างๆ ที่เป็นแบบ 3 มิติ ให้เป็น 2 มิติเมื่อเทียบกับขนาดของ Back Buffer หรือจอภาพ หลังจากนั้นก็จะได้ภาพปรากฏอยู่ใน Back Buffer เพื่อที่จะได้ส่งให้ Direct3D Device Object จัดการนำข้อมูลใน Back Buffer นี้ไปสลับหรือ Copy เป็น Front Buffer เพื่อให้ภาพไปปรากฏให้เห็นบนจอภาพต่อไป

การคำนวณเรื่อง Transformation และ Lighting นี้จะใช้เวลานาน และกินแรง CPU มาก เพราะต้องใช้ส่วน FPU (Floating Point Unit) มาช่วยคำนวณเรื่อง Vertex ต่างๆ ซึ่งถ้าหากการ์ดเร่งความเร็วสามมิติใด Support การทำงานในส่วนนี้ก็จะทำให้ช่วยแบ่งเบาการทำงานของ CPU ไปได้เยอะ ทำให้ CPU มีเวลาไปคำนวณในส่วนอื่นๆ ผลก็คือโปรแกรมทำงานได้เร็วขึ้นมาก

3.6.6 World Transformation Matrix

เมทริกใน Direct3D จะเป็น Array ขนาด 4×4 ใช้สำหรับจัดการกับ Vector ต่างๆ ของ Vertex ซึ่งค่าที่อยู่ใน Array ของเมทริกนี้ไม่จำเป็นต้องรู้หรอก (ถ้าอยากรู้ก็ศึกษาเพิ่มเติมเอาเองนะ อ้อ) เพราะในไคลเรอ์ SDK มันมีฟังก์ชันง่ายๆ ในชุด D3DX Library ให้เรียกใช้ ซึ่งการคำนวณเมทริกจะต้องสร้าง Object ซึ่งเป็นตัวแปรแบบ Structure ขึ้นมาเสียก่อน (Structure ที่ใช้ก็คือ D3DXMATRIX)

World Transformation เมทริกเป็นเมทริกที่ใช้สำหรับนำโมเดลต่างๆ ไปไว้ใน World Space คือย้ายจาก Model Space ไปสู่ World Space ซึ่งขณะที่ทำการย้ายนั้น สามารถหมุน, ย่อ, ขยาย หรือวางโมเดลแต่ละอันไว้ที่ตำแหน่งใดใน World Space ก็ได้ โดยที่ไม่ต้องไปแตะต้องค่าของตำแหน่งดั้งเดิมของ Vertex ต่างๆ ที่กำหนดไว้ในโมเดลแต่ละอันเลย

กฎหรือหลักการแปลงจาก Model Space มา World Space ที่ดีก็คือ ถ้าอยากทำอะไรกับโมเดลแต่ละอันก็ทำไปก่อนแล้วเก็บค่าต่างๆ ไว้ในเมทริกธรรมดาที่สร้างขึ้นไว้ จากนั้นจึงค่อยรวมเมทริกแต่ละอันนั้น ให้เป็นเมทริกผลลัพธ์รวมอันเดียว แล้วจึงสั่งให้ทำกระบวนการ World Transformation ด้วยการใส่เมทริกผลลัพธ์นี้เป็นหลัก ซึ่งทำได้โดยการเรียกใช้ เมธอด ชื่อ IDirect3DDevice8::SetTransform() โดยส่ง Parameter ตัวแรกเป็นค่าคงที่ชื่อ D3DTS_WORLD และ Parameter ตัวสุดท้ายเป็น Address ของเมทริกผลลัพธ์ดังกล่าว

3.6.7 View Transformation Matrix

เป็นเมทริกที่ใช้สร้างกล้อง (Camera) ซึ่งมันจะจับภาพบางส่วนใน World Space หรืออีกในหนึ่งก็คือ แปลงหรือตำแหน่งของโมเดลต่างๆ ที่อยู่ใน World Space ตรงตำแหน่งที่กล้องจับภาพอยู่ มาเป็นของ Camera Space

และกฎที่สำคัญ รวมทั้งฟังก์ชันที่ใช้ในการหมุน, เคลื่อนย้าย หรือปรับขนาดของมุมมองของกล้องก็เหมือนกับที่ใช้กับ World Transformation เมทริกยกเว้นเรื่องที่ว่า ต้องมีการบอกให้ Direct3D รู้ด้วยว่า จะใช้ 3DSystem แบบ Left-Hand หรือ Right-Hand โดยใช้ฟังก์ชัน D3DXMatrixLookAtLH() หรือ D3DXMatrixLookAtRH() ตามลำดับ ซึ่งจะใช้แบบมาตรฐานของ Direct3D คือ Left-Hand สำหรับ

Parameter ที่ต้องส่งไปให้ฟังก์ชันนี้มีอยู่ 4 ตัว ตัวแรกเป็น Address ของเมทริกที่จะใช้เก็บผลลัพธ์ที่ได้ ตัวที่สองเป็น 3D Vector เพื่อบอกว่า ตำแหน่งของกล้องอยู่ที่ไหน

จากนั้นสร้าง View Transformation เมทริกโดยการเรียกใช้ เมธอด เหมือนที่ใช้กับ World Transformation เมทริกคือ IDirect3DDevice8::SetTransform() แต่ให้ใช้ค่าคงที่สำหรับใช้เป็น Parameter ตัวแรกว่า D3DTS_VIEW แทน แล้วส่ง Address ของเมทริกผลลัพธ์ที่ได้จาก D3DXMatrixLookAtLH() เป็น Parameter ตัวสุดท้าย

จะเห็นว่า เอาส่วนการสร้าง View Transformation เมทริกไปไว้ในฟังก์ชัน fnInitGeometry() แทนที่จะเอาไปไว้ใน fnRender() ที่เป็นเช่นนี้ก็เพราะกล้องมันอยู่กับที่ตลอดเวลา ไม่เหมือนกับกรณี World Transformation ที่อาจมีการเปลี่ยนตำแหน่งของโมเดลในแต่ละเฟรม (นอกเสียจากว่าอยากเคลื่อนย้ายกล้องไปทั่วฉาก) ดังนั้นจึงไม่มีความจำเป็นที่จะต้องสั่งให้ Set ค่าเพื่อทำ View Transformation ทุกๆ เฟรม จึงทำครั้งแรกครั้งเดียวก่อนที่จะเข้า Loop การ Render

3.6.8 Projection Transformation Matrix

เป็นเมทริกที่ใช้สร้างมุมมองแบบ Perspective จากมุมมองของกล้องหรือของ Vertex ต่างๆ ที่อยู่ใน Camera Space โดยจะแปลงให้มาอยู่ใน Projection Space ซึ่งวิธีสร้างจะต้องใช้ฟังก์ชันแบบ Left-Hand หรือ Right-Hand คือ D3DXMatrixPerspectiveFovLH() กับ D3DXMatrixPerspectiveFovRH() ตามลำดับ แต่จะใช้แบบ Left-Hand โดย Parameter ตัวแรกที่ส่งให้ฟังก์ชันนี้ก็คือ Address ของเมทริกที่จะใช้เก็บผลลัพธ์ที่ได้ ส่วน Parameter ตัวที่สองจะเป็นมุมแบบเรเดียนซึ่งหมายถึงระยะทางของมุมมองที่จะทำ Perspective หรือที่เรียกว่า Field of View (FOV) ถ้ามุมกว้างมาก จะหมายถึงการดึงภาพกว้างมาก มาบีบให้แสดงในพื้นที่ของหน้าจอ ผลก็คือเกิด Perspective มากเมื่อมองโมเดลแบบใกล้ๆ (ภาพจะเพี้ยนมาก ทำให้เวียนหัวและคลื่นไส้อยากจะอา้วก) ปกติมักจะใช้ 45 องศา หรือ $\pi / 4$ เรเดียน (สามารถใช้ค่าคงที่ D3DX_PI แทนค่า π ได้) ในเกม Half-Life หรือ Half-Life: Counter-Strike จะใช้ถึง 72 องศา หรือ $\pi / 2.5$ เรเดียน